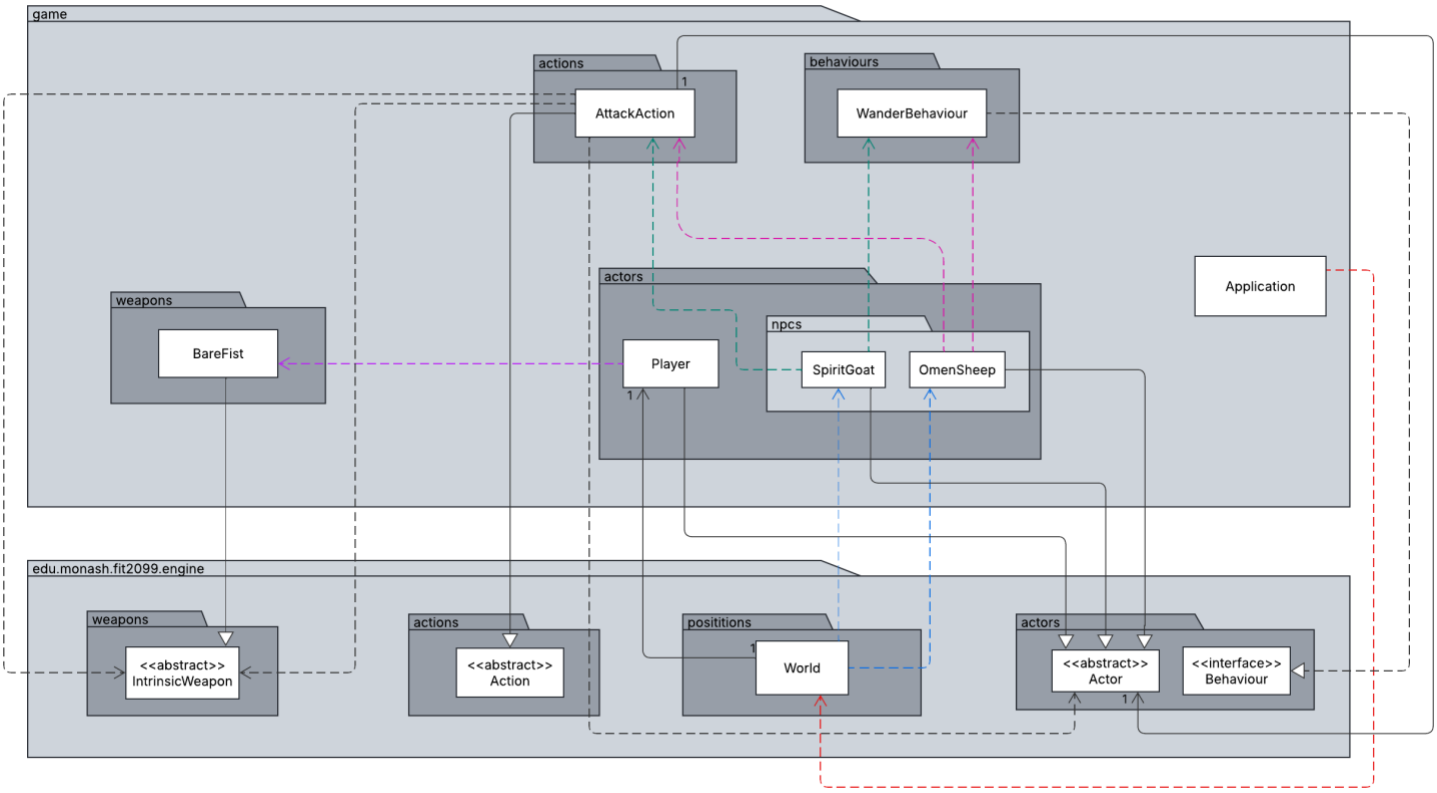
FIT2099 A1 Design Documentation

REQ 1



The UML diagram represents an extension to the existing Elden Thing game architecture to support non-hostile NPCs (SpiritGoat and OmenSheep) that can:

1. Move independently around the game world
2. Be attacked by the player

<u>SpiritGoat and OmenSheep Class</u>

The design introduces two non-hostile creature types SpiritGoat and OmenSheep. Both NPCs extend the abstract Actor class to inherit base capabilities such as allowable actions, behaviours, and inventory. This aligns with LSP as these classes can be used anywhere an Actor is expected.

To differentiate them from enemy actors, both are marked with a custom enumeration Status.NON_HOSTILE_TO_ENEMY. This uses the existing capability system to tag and process actors differently, following OCP. We extend functionality by tagging them with a new enum value, without modifying existing logic.

Each class assigns its own displayChar, hitPoints, and behaviours. However, the design violates DRY because the only differences are visual and numerical properties. Ideally, a shared superclass like PassiveCreature would be created to consolidate common logic, but Java's single inheritance restriction limits this unless composition or delegation is used.

WanderBehaviour Class

Both creatures are given autonomous movement through WanderBehaviour, which implements the Behaviour interface. This cleanly encapsulates pathfinding logic in a dedicated component, achieving SRP and ISP. The design uses a Map<Integer, Behaviour> for prioritising behaviours—this provides flexibility in adding new behaviour layers without modifying core logic (OCP). It is reusable and modular across different Actor types.

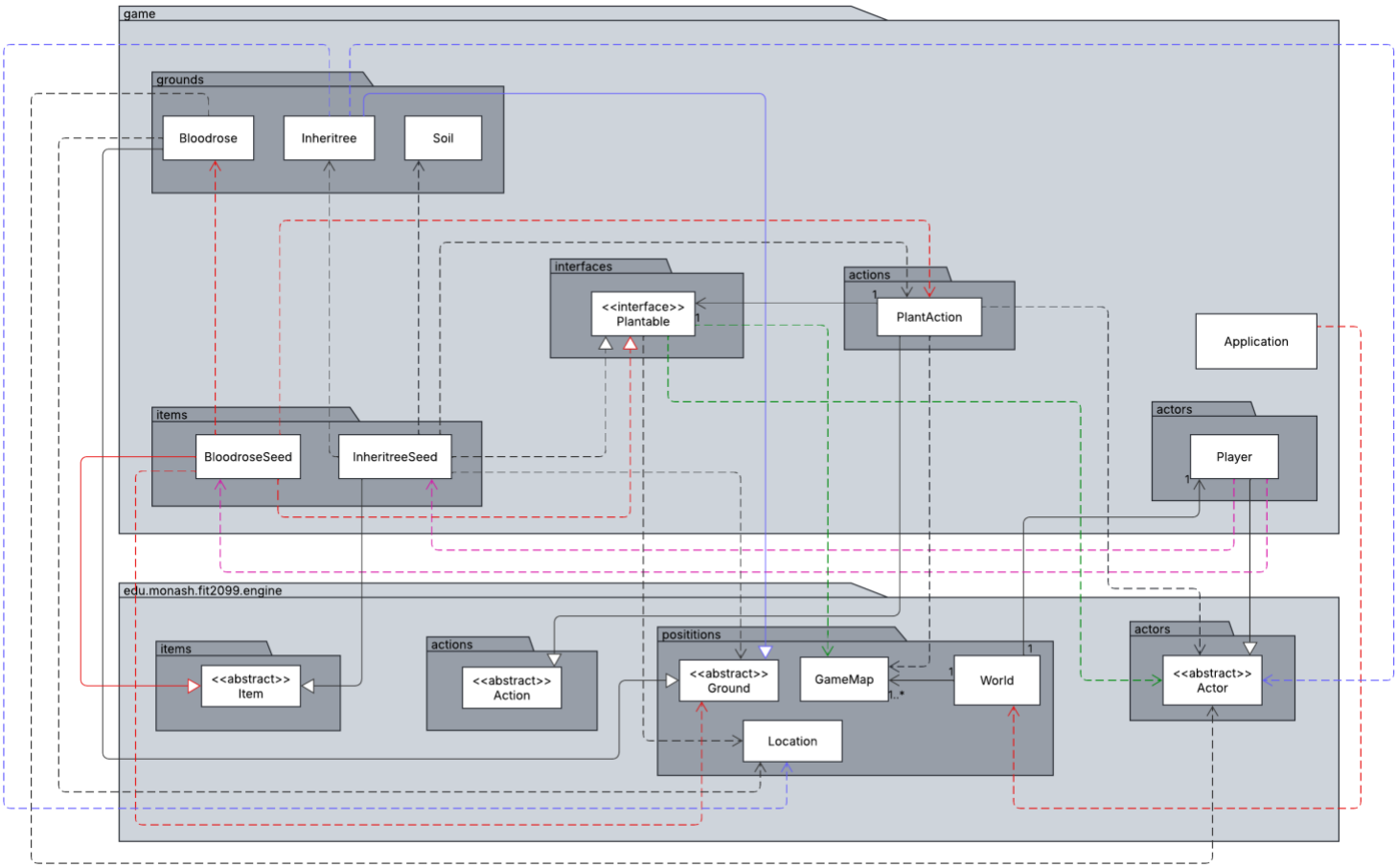
AttackAction Class

The dependency on AttackAction ensures these NPCs can be attacked. The player can engage with any Actor as they as marked with Status.HOSTILE_TO_ENEMY. Because AttackAction operates generically on Actor, LSP is upheld—SpiritGoat and OmenSheep are treated like any other actor in combat. Additionally, AttackAction follows SRP by encapsulating the logic for inflicting damage.

Status.NON_HOSTILE_TO_ENEMY Enumeration

SpiritGoat and OmenSheep classes are marked with Status.NON_HOSTILE_TO_ENEMY to distinguish them from hostile entities, allowing the game engine to process them differently during combat and interaction phases. This approach leverages the existing status system to clearly mark entities that should not initiate attacks against the player, while still allowing other interactions to occur.

REQ 2



The UML diagram represents an extension to the Elden Thing game architecture that enables the Farmer to plant crops on soil. This system introduces two different crop types with distinct behaviours:

1. Inheritree:
    - Transforms surrounding blighted terrain into fertile soil
    - Provides healing effects to nearby Actors each turn
2. Bloodrose:
    - Inflicts damage to Actors in its surrounding area each turn

Plantable Interface

The design centres around a Plantable interface implemented by seed classes (InheritreeSeed and BloodroseSeed), establishing a clear contract for items that can be planted. The Plantable interface provides the mechanism to transform a seed into its corresponding crop form. This approach adheres to the ISP and SRP by defining a focused contract specifically for plant-able behaviour without forcing unrelated methods on implementing classes. It also follows OCP as new seed types can be added by implementing the Plantable interface.

Design alternatives to the Plantable interface I considered included a Status-based approach where Status.SEED could be used to track planting state. However, this was rejected due to poor cohesion as each seed would require its own implementation method. It would also violate SRP by mixing state tracking with behaviour implementation. I also considered extending the abstract Item class to create a PlantableItem abstract class. However, this was rejected due to:

- Heavier implementation compared to interfaces

- Multi-level inheritance limitations in Java
- Reduced flexibility for classes that might need to inherit from other hierarchies

PlantAction Class

PlantAction serves as the bridge between seeds and the environment. I extended the Status enumeration to include FERTILE and added Status.FERTILE to the Soil class to indicate that it is allowed to be planted on. Coupled with the association between PlantAction and Plantable interface, this ensures that any class implementing Plantable can be planted on Soil.

This design choice adheres to the OCP as the Status enumeration can be extended without modifying existing code, while providing a clear semantic indication of soil readiness for planting. The Soil-PlantAction relationship leverages this status to enforce planting constraints, ensuring that seeds can only be planted on fertile ground. Together, this ensures behavioural consistency throughout the system and achieves LSP as any Plantable can be used where a seed is expected. This also adheres to DIP due to dependency on abstractions (Plantable interface), allowing for flexible substitution of different seed types while maintaining a single responsibility for the PlantAction class to handle the planting operation.

InheritreeSeed and BloodroseSeed Class

I implemented concrete seed classes that implement the Plantable interface, each with specific transformation logic to produce their respective crops when planted. This design maintains a clean separation between different seed types while allowing for specialised behaviour. The implementation follows the SRP by focusing each seed class on its specific crop transformation.

Advantages:

- Encapsulates seed-specific properties and behaviours
- Clear connection to corresponding crop types
- Follows OCP as new seed types can be added without modifying existing code

Disadvantages:

- Potential for duplicated code between seed implementations
- Each seed needs to manage its own transformation logic

Inheritree and Bloodrose Class

Inheritree and Bloodrose classes represent the crop that appears after planting their respective seeds. This implementation centralises their unique effects in their own classes, making it easier to understand and modify without affecting other parts of the system.

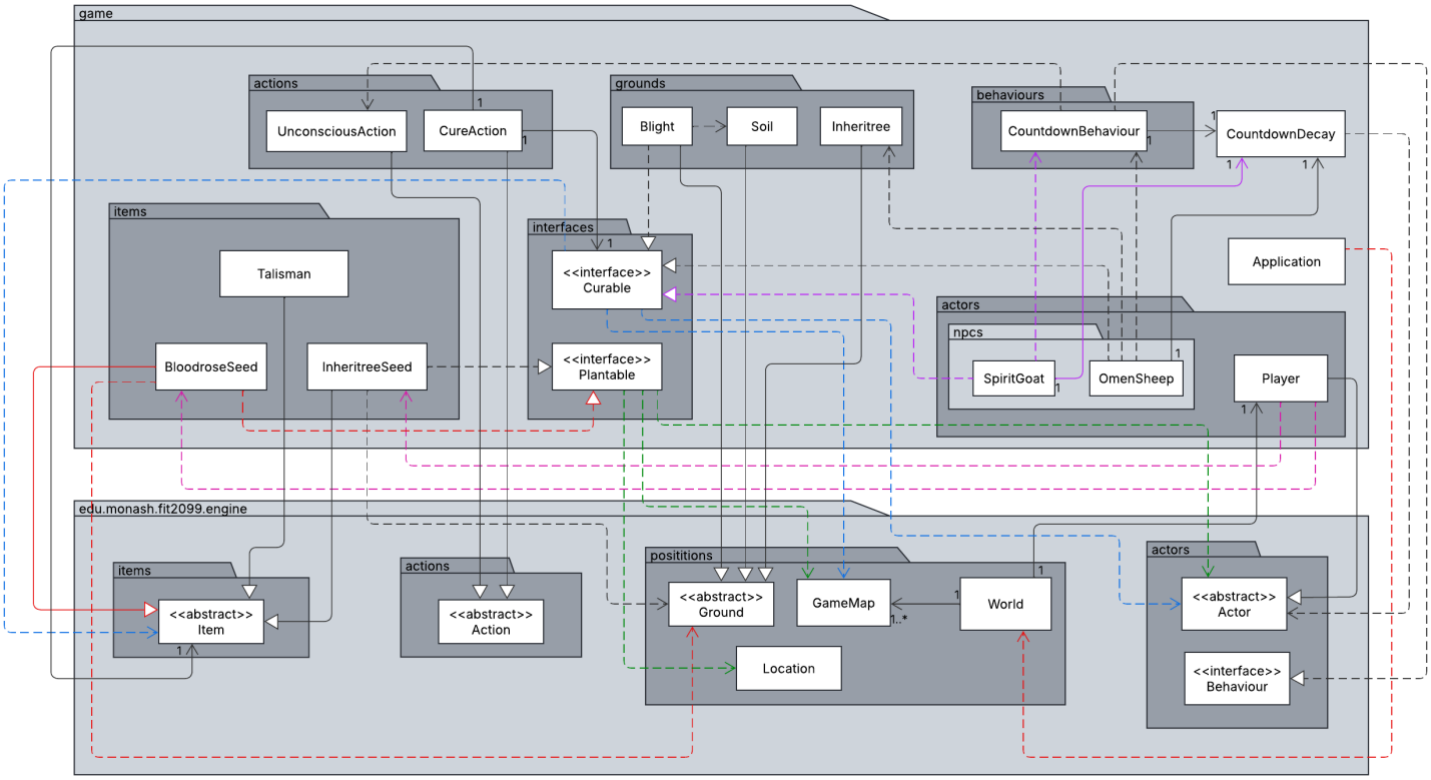
Advantages:

- Encapsulates crop-specific effects and behaviour
- Clear separation between the seed and the resulting plant
- Follows SRP by focusing on crop-specific functionality

Disadvantages:

- Dependencies on both Actor and Ground classes could create coupling issues

REQ 3



The UML diagram represents two key gameplay extensions to the Elden Thing game architecture:

1. Countdown system where NPCs slowly decay due to the Crimson Rot, in line with the game's lore. Specifically, the SpiritGoat and OmenSheep NPCs disappear after a fixed number of turns.
2. Curing system where the Talisman item enables the player to cure various entities such as OmenSheep, SpiritGoat, and Blight.

<u>Countdown System</u>

Key Classes:

- CountdownDecay: Maintains state and logic for tracking turns remaining. This logic is extracted from NPC classes to comply with SRP, ensuring the actor isn't responsible for its own timed death.

- CountdownBehaviour: Implements the Behaviour interface to decrement the countdown and trigger removal. This allows the effect to be injected into the actor's behaviour list, supporting OCP and LSP (new behaviour added without modifying existing actor logic).

Advantages:

- Compositional design—any Actor can adopt this mechanic by associating a CountdownDecay and adding a CountdownBehaviour.
- Fully decoupled from concrete NPC types, making it reusable across other decaying entities.

Disadvantages:

- Only supports fixed-length countdowns without variance unless extended.

Extensibility:

- Future NPCs suffering different forms of decay (e.g. faster rot, conditional expiry) can extend CountdownDecay or plug in alternative countdown-based behaviours.

Curing System

Key Components:

- Curable Interface: Entities implement this interface to define how they react to being cured, adhering to SRP and LSP. Each entity determines its own cure logic via beCuredBy().
- CureAction Class: Coordinates the curing process by calling the Curable's method. Depends on the abstraction (Curable) rather than concrete classes, supporting DIP.
- Talisman Item + Status.CURATIVE: Extends the Status enum to mark curing-capable items. The Actor inventory is scanned for Status.CURATIVE items to populate CureActions. This is a lightweight and extensible tagging mechanism aligned with OCP.

Design Trade-Offs:

Currently, beCuredBy() only allows for one type of cure logic per entity. This limits flexibility—for instance, if multiple CURATIVE items should cure differently (e.g. Talisman purifies, HerbalRemedy restores health), the system cannot easily support this distinction.

One considered alternative was to give each item a Map<Class<? extends Curable>, CureEffect> to define specific cure logic. However, this would violate OCP and DIP by pushing knowledge of cure logic into the item itself and tightly coupling it with all possible Curable types.

Pros:

- Clear separation of concerns (ISP) via interface abstraction (Curable)
- Behaviour is extensible—new Curables or CURATIVE items can be introduced without modification
- Reuses existing systems (e.g., Status enum, Action system) effectively

Cons:

- Limited polymorphic flexibility in cure methods (1 fixed behaviour per Curable)
- UI clutter if many CURATIVE items are in inventory and target multiple Curables